Assignment 4: Non-line-of-sight imaging - Lab sessions 7 and 8 -

1 Introduction

The focus of this assignment is implementing a method to reconstruct non-line-of-sight (NLOS) geometry based on time-resolved measurements acquired at a relay wall. Your main task will be developing a NLOS reconstruction algorithm based on backprojection, testing it on different NLOS datasets, and to analyze its performance and ability to reconstruct hidden scenes under different geometric configurations and capture configurations (see Figure 1). In this assignment:

1. You will implement a naïve version of the backprojection NLOS reconstruction method described by Velten et al. [3]. Section 2 in this document contains a brief overview of the principal aspects of the method. Please refer to the original article and the lecture notes for more details on the method.
2. You will adapt your method to handle confocal measurements and compare the performance with respect to non-confocal measurements.
3. You will modify your method to mitigate part of the attenuation effects produced by distance falloff and cosine foreshortening and illustrate it in two scenes.

Note that you are not required to provide a high-performance implementation of the backprojection method. It's ok to nest multiple `for` loops as long as the method provides the correct reconstruction.

2 Backprojection method

In this task, you will implement a backprojection method to reconstruct NLOS geometry using the transient datasets provided in the supplemental material. You have to implement the basic algorithm introduced by Velten et al. [3] to reconstruct a hidden scene from an arbitrary number of laser and single-photon avalanche detector (SPAD) points on a relay wall. In particular, for every voxel centered at $\mathbf{x}_v$ on a hidden scene, a simple (unfiltered) backprojection reconstruction $G(\mathbf{x}_v)$ of that voxel from a set of laser-SPAD measurements H on the relay wall is defined as:

$$G(\mathbf{x}_v) = \sum_{\forall \mathbf{x}_l, \mathbf{x}_s} H(\mathbf{x}_l, \mathbf{x}_s, t_v) \tag{1}$$

where t_v corresponds to the time-of-flight of third-bounce light paths from the laser device to the relay wall positions $\mathbf{x}_l \in L$, to the reconstructed voxel $\mathbf{x}_v \in V$, back to SPAD positions on the wall $\mathbf{x}_s \in S$, and back to the SPAD device. You can find a schematic of these light paths in Figure 2.

2.1 Datasets

You can load the datasets in the `.mat` files of the dataset folder using the `load()` Matlab function. You can find the description of the output `struct` fields in the README file of the datasets folder. Figure 3 shows the four different hidden scenes whose NLOS measurements are provided in the datasets.

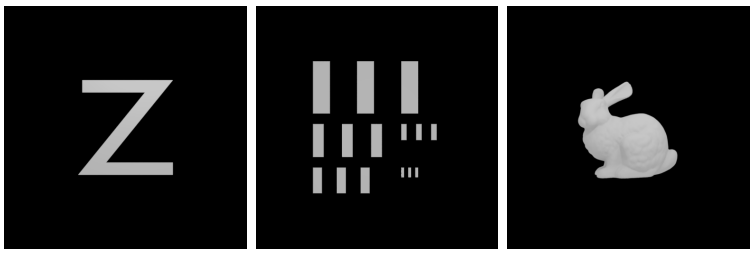

Figure 1: Illustration of the hidden scenes in the provided datasets, from left to right: `Z_*`, `usaf_*`, `bunny_*`.

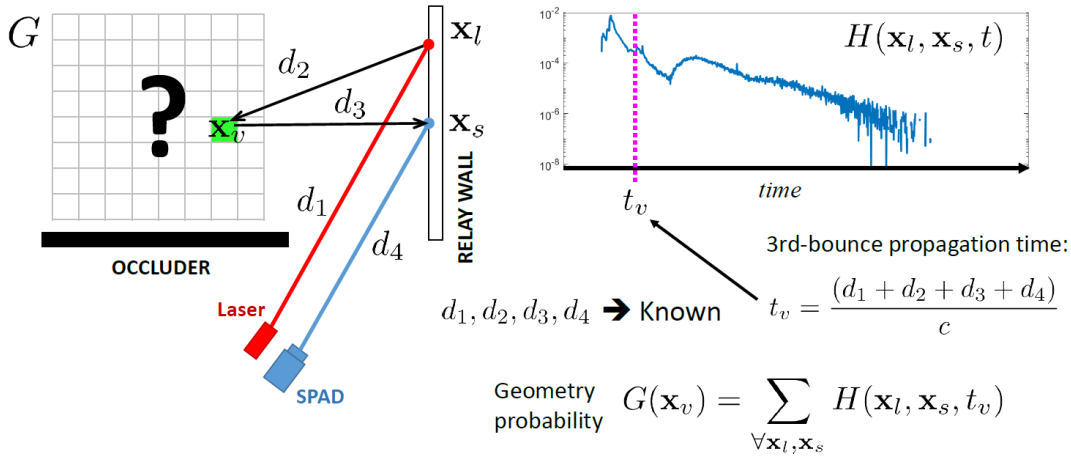

Figure 2: Schematic view of the backprojection reconstruction for a single voxel of the hidden scene.

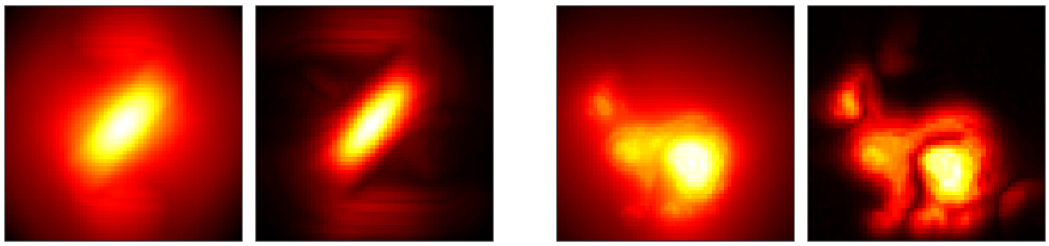

Figure 3: Results of the non-confocal Z and confocal bunny scene, obtained by performing a maximum intensity projection on a 3D reconstruction of size $64 \times 64 \times 64$ voxels. Left images correspond to unfiltered backprojection, right images were obtained by applying a Laplacian-of-Gaussian filter.

Temporal histograms Each simulated dataset contains a discretized impulse response functions H in the field `data`, in form of spatio-temporal histogram. This means that the temporal domain is discretized in multiple bins. Each bin represents the sum of all photons that arrived within an interval Δt, starting at t_0 (usually $t_0 = 0$) at the first bin of the temporal dimension, and advancing linearly—i.e. if $t_0 = 0$, the first bin gathers photons with time of flight $[0, \Delta t]$, the second $[\Delta t, 2\Delta t]$, etc. The Δt value is given in the `deltaT` field of the dataset in meter units, so you can simply operate in meters throughout the time-of-flight computations and avoid dividing by the speed of light. Take these aspects into account when computing all the necessary info to access values of H.

Relay wall time normalization The simulated datasets `usaf_d=0.5_l=[1x1]_s=[256x256].mat` and `bunny_d=0.5_c=[256x256].mat` are "time-normalized" at the relay wall. This means optical distances d_1 and d_4 have been already subtracted from all arriving photons when computing the discretized impulse response function H. **Do not include d_1 and d_4 when calculating the time of flight t_v in those datasets.**.

TO-DO: Load each dataset, and visualize a spatio-temporal slices ($x - t$ and $y - t$) of the impulse response function H (i.e. the `data` field) using the `imagesc` function and the `hot` colormap. Observe how, for datasets without time normalization, the wavefronts on the $x - t$ slices are temporally skewed on the x dimension.

2.2 Backprojection, visualization, and filtering

Implement the backprojection method, and test it over the `Z_d=0.5_l=[1x1]_s=[256x256].mat` and `usaf_d=0.5_l=[1x1]_s=[256x256].mat` datasets.

Visualizing a 2D projection Once you have a 3D reconstruction, you can visualize a front 2D projection of it (such as in Figure 3) by taking the maximum value over the depth dimension (or any other 2D projection using other dimensions), and displaying the resulting 2D image with `imagesc`.

Interactive 3D visualization Alternatively, to display an interactive 3D visualization of the reconstruction you can use the function `volshow` available in MATLAB along with the view configuration provided in the file `volshow_config.mat` to correctly display the reconstructed volume as `volshow(G,volshow_config)`. For newer versions of MATLAB (above 2020a), use the following set of instructions to obtain an equivalent visualization

```
vs_h = volshow(G, RenderingStyle="MaximumIntensityProjection", Colormap=hot);
vs_h.Parent.BackgroundColor = [0 0 0];
vs_h.Parent.GradientColor = [0 0 0];
```

You can also use the function `volumeViewer` of MATLAB, setting maximum intensity projection and the "hot" color map to have extra control over the visualization thresholds.

Filtering the reconstruction Apply a Laplacian or a Laplacian-of-Gaussian (LoG) filter to the reconstructed volumes to improve the quality of the reconstructions. Compare the results with and without filtering. You can use the following code snippet to e.g. apply a Laplacian filtering, assuming `G` is the 3D volume reconstructed with your backprojection method:

```
f_lap = fspecial3('lap');
G_lap = imfilter(G, -f_lap, 'symmetric');
G_lap(G_lap<0) = 0;
```

The Laplacian-of-Gaussian filter is more flexible than the Laplacian filter, allowing you to parameterize the standard deviation of the Gaussian. Check the documentation of the function `fspecial3` to learn how to use its parameters.

TO-DO: In your report, include visualizations of the reconstructed scenes from different viewpoints (e.g. 2D projections) at different resolutions of the voxelization and relay wall (e.g. using only a subset of points of H), and with and without filtering. Include the reconstruction time for each resolution you have used. Please answer the following questions and illustrate them with the obtained results when necessary:

1. How does the reconstruction time scale with respect to the hidden scene voxelization and the relay wall capture resolution?
2. Do you see any noticeable change in the reconstruction when using only a subset of points (e.g. 16x16, 32x32) of the relay wall?
3. Why do the outputs of unfiltered backprojection reconstructions look blurry? What is the effect of applying a Laplacian and Laplacian of Gaussian filtering over these outputs?

Some tips to consider when choosing reconstruction parameters:

- *Position and extent of the hidden geometry.* While in real applications, the extent of the target hidden geometry is unknown, the datasets in the supplemental material provide the position and dimensions of the region where the hidden geometry is contained. Namely, you can find this information in the dataset fields `volumePosition` and `volumeSize`. You can use this information to define the extent of your voxelization and avoid reconstructing large empty regions. To validate your implementation, we recommend you to reconstruct only the $x - y$ plane inside which the hidden object is contained.
- *Resolution of the voxelization and capture grids.* Choose a tractable volume resolution to test your algorithm. A reconstruction grid of size $4 \times 4 \times 4$ will be fast to compute but will yield a very coarse reconstruction. A reconstruction grid of size $1024 \times 1024 \times 1024$ will provide a finer reconstruction but the algorithm will take forever to finish. A voxelization with resolution of $8 \times 8 \times 8$ or $16 \times 16 \times 16$ could be a good starting point to test that your algorithm works correctly. A resolution of $32 \times 32 \times 32$ may be a good trade-off between reconstruction quality and execution time, once you know your method works well. You can also try skipping part of the laser and SPAD points on the relay wall (e.g., one every two, one every four, on each dimension) at the risk of affecting the reconstruction quality.

3 Backprojection on confocal measurements

Confocal capture topologies synchronously co-locate lasers and SPADs when scanning the relay wall, i.e. $\mathbf{x}_l \equiv \mathbf{x}_s$. Therefore, each illuminated point only generates one corresponding time-resolved measurement at the same location. This topology yields better results when surfaces in the hidden scene are not co-planar to the relay wall.

Reconstruct the non-confocal bunny `bunny_d=0.5_l=[1x1]_s=[256x256].mat` with your backprojection method. Then, add support to your method to handle confocal measurements, and apply it to the confocal version of the bunny scene `bunny_d=0.5_c=[256x256].mat`.

Include comparisons between the confocal reconstruction and the non-confocal reconstruction of the bunny scene. Is there any noticeable difference between the reconstructions?

4 Phasor-based filtering

Reconstruct the hidden scene using a phasor-based 1D Morlet wavelet filter applied over the temporal domain of the H function before performing the backprojection operation, and compare the results with other filtered reconstructions you did before. This involves implementing this variation of the backprojection equation:

$$G(\mathbf{x}_v) = \left| \sum_{\forall \mathbf{x}_l, \mathbf{x}_s} H'(\mathbf{x}_l, \mathbf{x}_s, t_v) \right| \tag{2}$$

where

$$H'(\mathbf{x}_l, \mathbf{x}_s, t_v) = H(\mathbf{x}_l, \mathbf{x}_s, t_v) *_t K_m(t), \tag{3}$$

where $*_t$ represents a convolution in the temporal domain, and

$$K_m(t) = e^{2j\pi\Omega_c t} e^{\frac{-t^2}{2\sigma^2}} \tag{4}$$

is a Morlet wavelet. This filtering strategy is based on phasor-field NLOS imaging methods [2, 1], and implements a complex-valued Morlet wavelet, i.e. a complex exponential with frequency Ω_c multiplied by a Gaussian envelope with standard deviation σ. To select values of Ω_c and σ follow this criteria:

- Choose $\Omega_c = 1/\lambda_c$ so that the wavelength λ_c is at least twice the spacing between points (either vertical or horizontal) on the relay wall. Check the SPAD and laser grids to retrieve this value. Note that if you downsample the input data to quickly test your implementation, the spacing between grid points will be larger, so you should choose λ_c according to the new spacing.
- Choose $\sigma \in [\frac{\lambda_c}{2\log 2}, 2\lambda_c]$.

Note this task not involve any major change of the backprojection algorithm. You only need to substitute the H evaluated in Equation (1) by its filtered counterpart H', and you only need to filter H once, before the actual algorithm starts. The indexing of the temporal domain in Equation (2) is the same as in Equation (1). Since the filtering introduces complex numbers in H, the reconstruction will also be complex-valued, so do not forget to apply an absolute value over the reconstruction before displaying it.

TIP 1 : Since the width of the temporal bins Δt in the datasets is defined in terms of optical distance (in meter units), we recommend you formulate the 1D array t and the rest of the parameters Ω_c, σ for the Morlet wavelet in meter units too, assuming the domain t represents the optical distance traveled by light instead of its time of flight. This makes it easier to define values for $\Omega_c = 1/\lambda_c$ and σ based on the distance between points in the relay wall.

TIP 2 : Performing a 1D convolution over the temporal domain of H using the `conv()` MATLAB function may require time-consuming iterations to perform individual temporal convolutions for every spatial coordinate of H. We recommend you apply convolutions as multiplications on the Fourier domain by first applying 1D Fourier transforms over the temporal dimension of H and K_m using `fft(variable, [], temporal_dimension)`, multiplying both transformed variables, and finally doing an inverse 1D Fourier transform `ifft(variable, [], temporal_dimension)` over the temporal domain to obtain the filtered H'.

5 Deliverable

Your deliverable should be an archive file (e.g. a ZIP file) with the following items:

1. All your MATLAB code, including commented code as well as a README file explaining how to use the code. Comment your design choices in the code, if any.
2. An up-to five page PDF report with all the results (both intermediate and final) obtained, the explanations on how you got your results, and why you decided to use certain parameters or methods and not others.

6 Evaluation

The maximum score possible for this assignment is 10 points, but note that all parts sum to 12, so you can still get 10 points even if you do not get the maximum grade in all the parts:

Datasets (Section 2.1)	1
Backprojection, filtering, and visualization (Section 2.2)	6
Backprojection on confocal measurements (Section 3)	3
Phasor-based filtering (Section 4)	2
Total	12

References

[1] Xiaochun Liu, Sebastian Bauer, and Andreas Velten. "Phasor field diffraction based reconstruction for fast non-line-of-sight imaging systems". In: Nature communications 11.1 (2020), pp. 1–13.

[2] Xiaochun Liu et al. "Non-Line-of-Sight Imaging using Phasor Fields Virtual Wave Optics". In: Nature (2019).

[3] Andreas Velten et al. "Recovering three-dimensional shape around a corner using ultrafast time-of-flight imaging". In: Nature Communications 3.1 (2012), p. 745. DOI: 10.1038/ncomms1747.